

Transformer review and quiz



hw13_solution.pdf
PDF



hw09_solution.pdf
PDF

Sultan 3-4pm 136 Social Sciences (Transformers/Attention)

HW9 Q4: Transformer Decoding Optimization

HW9 Q2: Argmax Attention

Joey 4-5pm Hearst Field Annex B1 (Post-training)

Review of RLHF and DPO Motivation

HW 13 Q2 (DPO)

HW 12 Old Exam Problem (if time permits)

Lance 5-6pm 242 Hearst Gym (Diffusion)

HW13 Q4: Diffusion Models

These are review sessions I am attending today. I have attached all the homeworks they are explaining problems from. Please clearly explain to me the concepts I need to know to be able to understand the review sessions. Add some questions at the end for me to answer as a quiz.

Let's do a "pre-review for your review sessions." I'll go by session, then give you a quiz at the end.

1. Sultan (Transformers / Attention)

1.1 Core transformer / attention recap

You should be comfortable with:

- **Token embeddings + positional encodings:**
Input tokens $\rightarrow$ vectors in $\mathbb{R}^{d_{\text{model}}}$, plus position info so the model knows order.
- **Multi-head self-attention (MHA):**
 - For each position, you compute

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where $X \in \mathbb{R}^{\text{seq} \times d_{\text{model}}}$.

- Attention weights for each head:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- Each **head** has its own W_Q, W_K, W_V , so each head can focus on different patterns. Heads are concatenated then projected with W_O . hw09_solution
- **Why the $1/\sqrt{d_k}$?**
Dot products of random d -dim vectors have variance $O(d)$; dividing by $\sqrt{d}$ keeps logits at a reasonable scale so softmax doesn't saturate. hw09_solution

1.2 HW9 Q2 – Argmax Attention

Standard attention uses **softmax** → a *smooth* probability distribution over keys. Argmax attention is a thought experiment:

- Compute similarities $s_i = q^\top k_i$.
- Instead of $\text{softmax}(s)$, use one-hot:
 - Let $j = \arg \max_i s_i$.
 - Output = value v_j (or one-hot vector times V). hw09_solution

Key conceptual point: why this is bad for training.

- **Softmax attention:**
 - Weights $w_i = \text{softmax}(s)_i$ are differentiable in the scores s_i .
 - Small change in any k_i or q smoothly changes all w_i → gradients flow to *all* keys, queries, and values.
- **Argmax attention:**
 - The mapping from scores → one-hot is *piecewise constant*; almost everywhere, changing q or a k_i **doesn't change the argmax index**.
 - That means gradients of the attention output w.r.t. most parameters are **zero almost everywhere**.
 - Only the selected head/key/value gets gradient (and even that is basically zero because argmax itself is non-differentiable).
→ You can't effectively learn good keys/queries/values with standard backprop; the model would "freeze" its attention pattern. hw09_solution

Takeaway for section: **Softmax is crucial for learning**, even if argmax might seem closer to a "hard lookup."

1.3 HW9 Q4 – Transformer Decoding Optimization (MHA vs MQA)

This question is about *incremental decoding* and how to make it efficient.

1.3.1 Incremental decoding with MHA

At generation step t :

- You have **current token embedding** $x \in \mathbb{R}^{b \times d}$.
- You have cached **past keys/values**:
 - $\text{prev_K} \in \mathbb{R}^{b \times h \times n \times k}$
 - $\text{prev_V} \in \mathbb{R}^{b \times h \times n \times v}$
- You compute new q, k, v per head:
 - Using `einsum("bd,hdq->bhq", x, W_q)`, etc.  hw09_solution
- Append new k, v to caches along the sequence dimension ($n \rightarrow n+1$).
- Compute attention weights over all $n+1$ positions for each head:
 - logits shape: $[b, h, n + 1]$
 - `weights = softmax(logits, dim=-1)`
- Weighted sum with V per head $\rightarrow$ concat $\rightarrow$ project with W_o .

Complexity intuition (per step):

- Compute projections (q, k, v, o): $\approx O(bd^2)$.
- Attention (multiply q with all cached keys, etc.): $\approx O(bnd)$.
- Memory access: reading weights, current activations, and **all cached K/V**, which is $\approx O(bnd)$.



hw09_solution

As context length n grows, you become **memory-bandwidth limited** because of those per-head cached keys/values.

1.3.2 Multi-Query Attention (MQA)

MQA's idea: *multiple heads share the same keys and values*.

- Shapes change:
 - Keys:
 - MHA: $W_k \in \mathbb{R}^{h \times d \times k} \rightarrow$ per-head keys.
 - MQA: $W_k \in \mathbb{R}^{d \times k}$, and cached $\text{prev_K} \in \mathbb{R}^{b \times n \times k}$.  hw09_solution
 - Values:
 - MHA: $W_v \in \mathbb{R}^{h \times d \times v}$.
 - MQA: $W_v \in \mathbb{R}^{d \times v}$, cached $\text{prev_V} \in \mathbb{R}^{b \times n \times v}$.
 - Queries stay per-head: $W_q \in \mathbb{R}^{h \times d \times q}$.
- Computation:
 - q: `torch.einsum("bd,hdq->bhq", x, W_q)  $\rightarrow$  [b, h, q]`
 - k: `torch.einsum("bd,dk->bk", x, W_k)  $\rightarrow$  [b, k]` (shared across heads)
 - v: `torch.einsum("bd,dv->bv", x, W_v)  $\rightarrow$  [b, v]` (shared across heads).  hw09_solution
- New caches:

- $\text{new_K} \in \mathbb{R}^{b \times (n+1) \times k}$
- $\text{new_V} \in \mathbb{R}^{b \times (n+1) \times v}$
- Attention:
 - `logits = einsum("bhk,bnk->bhn", q, new_K)`
 → **heads attend to the same K/V** but produce different mixtures (different q).

Complexity changes:

- **Compute** is still $O(bd^2 + bnd)$ in big-O, because:
 - still compute per-head q,
 - but only one K and V.
- **Memory**: reading cached K/V is now $O(bnk)$ instead of $O(bnd)$ because K/V are not multiplied by h. This is the big win. 

Intuition for the review:

For long sequences, decoding speed is often bottlenecked by memory bandwidth.

MQA reduces the size of the K/V cache by $\approx 1/h$, which makes decoding significantly faster and more scalable, while keeping multiple query heads for expressivity.

2. Joey (Post-Training: RLHF & DPO)

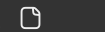
2.1 RLHF pipeline and motivation

Standard **RLHF (Reinforcement Learning from Human Feedback)** pipeline:

1. **Pretraining**: generic next-token prediction on massive text.
2. **SFT (Supervised Fine-Tuning)**:
 - Collect “good” responses for prompts.
 - Train model $\pi_{\text{SFT}}(y|x)$ with supervised loss.
3. **Reward modeling**:
 - For each prompt x , use π_{SFT} to sample several candidate answers.
 - Humans label preferences: winner y_w vs loser y_l .
 - Assume a latent reward $r^*(x, y)$ such that humans pick y_w with probability:

$$p(y_w \succ y_l | x) = \sigma(r^*(x, y_w) - r^*(x, y_l))$$

(Bradley-Terry model).

- Learn parametric reward model $r_\phi(x, y)$ by maximizing likelihood (equivalently, minimizing negative log-loss). 

4. RL fine-tuning with PPO:

- Optimize policy π_θ to maximize expected reward, but with a **KL penalty** to stay close to a reference policy (usually π_{SFT}):

$$\max_{\pi} \mathbb{E}_{x \sim D, y \sim \pi(y|x)} [r_{\phi}(x, y) - \beta \text{KL}(\pi(y|x) \parallel \pi_{\text{ref}}(y|x))].$$

- Hard because y is discrete $\rightarrow$ use RL (PPO).  hw13_solution

Motivation problems:

- Training a separate reward model is expensive.
- PPO RL is unstable and computationally heavy.
- Reward model can be mis-specified $\rightarrow$ reward hacking.

2.2 HW13 Q2 – Deriving DPO

This question walks you through a clean derivation of **Direct Preference Optimization (DPO)**.

Core steps:

1. Solve the constrained KL-regularized RL problem in closed form.

For fixed reward $r(x, y)$ and reference policy π_{ref} , the optimal policy is:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right),$$

where $Z(x)$ is a partition function. This comes from rewriting the objective as minimizing a KL divergence between π and that Boltzmann-reweighted policy.  hw13_solution

2. Re-express the reward in terms of π^* and π_{ref} :

From the above relation:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

3. Plug this into the Bradley-Terry model:

- BT gives:

$$p(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)).$$

- When you substitute the expression for r , the **$Z(x)$ term cancels**:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}.$$

- So preferences can be written directly in terms of **log-ratios of policy vs reference**; no partition function.  hw13_solution

4. Define a parametric policy π_{θ} and form a maximum-likelihood objective over preference data:

The DPO loss becomes:

$$L_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

$\rightarrow$ This is just a **binary cross-entropy loss** where the model should assign higher “implicit reward” to the winner than to the loser.

5. Gradient intuition:

The gradient looks like:

$$\nabla_{\theta} L_{\text{DPO}} \propto \sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w)) (\nabla_{\theta} \log \pi_{\theta}(y_w|x) - \nabla_{\theta} \log \pi_{\theta}(y_l|x))$$

where $\hat{r}_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$. hw13_solution

- If the model is **correct** (winner has much higher implicit reward), then $\hat{r}(y_l) - \hat{r}(y_w) \ll 0$, $\sigma \approx 0 \rightarrow$ small gradient.
- If the model is **wrong** (loser rated higher), $\sigma \approx 1 \rightarrow$ big gradient.
 $\rightarrow$ DPO focuses updates on **examples it's mis-ranking**.

Big picture: DPO lets you **skip explicit reward modeling and RL**; you just train the policy itself with a supervised-style loss on pairwise comparisons, while still implementing a KL-regularized reward improvement.

2.3 HW12 old exam problem

You don't have the PDF here, but conceptually it's likely testing:

- Understanding the **three RLHF phases** (SFT, reward modeling, RL fine-tuning).
- Why **KL regularization** to a reference policy is important (don't drift; maintain language quality).
- How **pairwise preference data** yields a reward model (Bradley-Terry).
- How post-training methods relate (SFT vs RLHF vs DPO).

If you're solid on those bullet points and HW13 Q2, you'll be fine.

3. Lance (Diffusion — HW13 Q4/Problem on Diffusion Models)

This problem uses a toy 1D Gaussian example to illustrate diffusion ideas (DDPM & DDIM). hw13_solution

3.1 Forward diffusion process

- Data distribution: $X_0 \sim \mathcal{N}(0, \sigma^2)$ with small σ^2 .
- Forward process: for t from 0 to 1 (continuous time), keep adding tiny Gaussian noise with variance Δt at each step.

In this simplified model:

- After time t :
 $X_t \sim \mathcal{N}(0, \sigma^2 + t)$.
- At $t=1$:
 $X_1 \sim \mathcal{N}(0, \sigma^2 + 1) \approx \mathcal{N}(0, 1)$ since $\sigma^2 \ll 1$.

So the forward process gradually **destroys structure** by adding noise until you get (almost) pure standard normal noise. hw13_solution

3.2 Reverse diffusion and why you need noise

They compute the conditional distribution of $X_{t-\Delta t} | X_t$:

- Mean: almost a scalar times X_t .
- Variance: $\approx \Delta t$ (small but nonzero). 
hw13_solution

Now imagine doing reverse steps **without** adding stochastic noise:

- At each step, just map to the conditional mean.
- You can express $X_0^{(\text{back})}$ as a product of scalars times X_1 ; the variances multiply.
- The telescoping product gives something like:

$$X_0^{(\text{back})} \sim \mathcal{N}\left(0, \left(\frac{\sigma^2}{\sigma^2 + 1}\right)^2\right) \approx \mathcal{N}(0, \sigma^4).$$

For tiny σ^2 , σ^4 is *much* smaller $\rightarrow$ the samples collapse around 0, way too low variance. 
hw13_solution

Lesson:

If you try to “denoise deterministically” with no randomness, your generated distribution is too narrow. In real DDPMs, you *must* inject noise in the reverse process.

When you add back appropriate Gaussian noise in each reverse step, they show that the final variance recovers σ^2 (up to tiny corrections): the reverse chain is a valid sampler for the target distribution.


hw13_solution

3.3 Training the score / denoiser network

In the toy problem, the ideal reverse step uses:

$$g(x_t, t) = \mathbb{E}[X_0 | X_t = x_t],$$

which in the Gaussian case has a **known analytic form** (a shrinkage factor times x_t). But in general we don’t know it.

So they describe how to **train a neural net** $f_\theta(x_t, t)$ to approximate $g(x_t, t)$:

- Sample:
 - x_0 from the dataset.
 - t from Uniform[0,1].
 - noise $z \sim \mathcal{N}(0, t)$.
- Compute $x_t = x_0 + z$.
- Input to net: (t, x_t) .
- Target: x_0 .
- Loss: mean-squared error $\|x_0 - f_\theta(t, x_t)\|^2$. 
hw13_solution

This is the “denoising score matching” perspective: learn to **recover clean data from noised data** at various noise levels.

3.4 DDPM vs DDIM

- **DDPM:**
 - Reverse process is stochastic: conditional mean + noise.
 - More “random” samples; good coverage.
- **DDIM:**
 - Removes randomness from reverse steps but adjusts the step size (ODE view).
 - They approximate a deterministic update:

$$x_{t-\Delta t}^{\text{DDIM}} \approx x_t \left(1 - \frac{\Delta t}{2(\sigma^2 + t)} \right)$$

(roughly). 

- When you multiply all these contractions across steps and take a continuous limit, you still get the right variance ($\approx \sigma^2$) if you choose them carefully. 
hw13_solution

Conceptual takeaway for the session:

Diffusion models gradually add Gaussian noise in the forward process and learn a network that approximates the reverse process. DDPM uses noisy reverse steps; DDIM uses deterministic steps but with a carefully chosen contraction schedule so you still end up with plausible samples.

4. Mini-Quiz for You

Try these on your own; if you want, you can send answers and I'll check.

4.1 Transformers / Attention

1. In standard multi-head attention, why do we divide the dot product $QK^\top$ by $\sqrt{d_k}$ before applying softmax?
2. Suppose you replace softmax in attention with argmax.
 - (a) What does the attention output become?
 - (b) Why is this bad for gradient-based training?
3. In incremental decoding with MHA, what are `prev_K` and `prev_V` used for, and why do we cache them instead of recomputing from scratch each step?
4. In MQA, why can keys and values be shared across heads without losing the benefit of having multiple heads?

4.2 RLHF & DPO

5. Briefly describe each stage of the RLHF pipeline and its purpose: SFT, reward modeling, RL fine-tuning.
6. In the Bradley-Terry model, why does it make sense to use $\sigma(r(x, y_w) - r(x, y_l))$ as the probability that humans prefer y_w over y_l ?

7. Show (in words, not detailed algebra) why the partition function $Z(x)$ cancels when you plug the optimal policy into the Bradley-Terry model.
8. In the DPO loss, what does the term

$$\nabla_{\theta} \log \pi_{\theta}(y_w|x) - \nabla_{\theta} \log \pi_{\theta}(y_l|x)$$

do to the policy, intuitively?

9. Why does DPO apply **larger** gradient updates when the model assigns higher “implicit reward” to the **loser** than to the winner?

4.3 Diffusion

10. In the toy Gaussian diffusion example, why does the forward process have $X_t \sim \mathcal{N}(0, \sigma^2 + t)$?
11. Why does doing reverse diffusion with only the conditional mean (no noise) give a distribution whose variance is *too small*?
12. Conceptually, how do you generate training data for the denoiser network $f_{\theta}(x_t, t)$? What are the inputs and targets?
13. What’s the key difference between DDPM and DDIM in how they perform the reverse process?

If you want, next step I can do: super short “one-pager” crib notes just for DPO or just for MQA/argmax attention that you can glance at right before each session.

Sources

We divide the dot product QK^T by the square root of dk so that the logits going into softmax stay in a reasonable numeric range as dk grows. Without this scaling, the dot products grow in magnitude with dk , softmax saturates (very peaky or flat), and gradients become tiny or unstable; the scaling keeps them roughly order 1 and stabilizes training.

- a) With argmax instead of softmax, each query chooses exactly one key: the one with the largest dot product. The attention output becomes the single corresponding value vector (a hard one-hot choice instead of a weighted average).
- b) Argmax is basically non-differentiable, so gradients with respect to the attention weights are zero almost everywhere. That kills gradient flow through the attention mechanism and makes end-to-end gradient-based training not work well.

In incremental (autoregressive) decoding, $prev_K$ and $prev_V$ store the keys and values for all previously generated tokens. At a new time step you only compute K and V for the new token and append them to $prev_K$ and $prev_V$, then attend over the concatenation. Caching them avoids recomputing keys and values for the entire prefix at every step, turning an expensive recomputation into a cheap update.

In multi-query attention, each head has its own query projection but all heads share the same keys and values. This works because keys and values form a shared “memory” of the sequence; the diversity between heads comes from using different queries that look at this same memory in different ways. You still get multiple perspectives from different queries, while sharing K and V saves memory and compute without removing the benefit of multiple heads.

RLHF stages:

- Supervised fine-tuning (SFT): start from the base language model and fine-tune it on high-quality human-written or human-edited examples. Goal: make the model generally helpful, safe, and instruction-following.
- Reward modeling: collect human preference data over pairs of model outputs, and train a reward model $r(x, y)$ that predicts which answer humans would prefer. Goal: turn human judgments into a scalar reward signal.
- RL fine-tuning: treat the language model as a policy and optimize its outputs to get high reward according to the reward model, usually with a KL penalty to stay close to the SFT model (e.g., PPO with KL regularization). Goal: directly align model behavior with human preferences.

In the Bradley-Terry model, each response y is assumed to have a latent “score” $r(x, y)$. The probability of preferring y_w over y_l should depend only on the score difference $r(x, y_w) - r(x, y_l)$. Using the logistic function σ of that difference is natural because it maps any real number to a probability between 0 and 1, is symmetric (swapping the two options flips the probability), and makes larger score differences correspond to stronger preferences.

Under the optimal policy, the probability of a response is proportional to $\exp(r(x, y))$, normalized by a partition function $Z(x)$. In the Bradley-Terry setup, you only care about ratios of probabilities for two responses, such as “probability of choosing y_w among y_w and y_l .” Both probabilities include the same $Z(x)$. When you form the ratio or difference that defines the preference probability, $Z(x)$ appears in every term and cancels out. Intuitively, the global normalization over all possible responses does not matter when you only compare a pair.

The term “gradient of $\log \pi$ for the winner minus gradient of $\log \pi$ for the loser” pushes the policy to increase the probability of the preferred response and decrease the probability of the dispreferred response. Intuitively, it shifts probability mass away from the loser and toward the winner so that future samples look more like y_w and less like y_l .

DPO scales that gradient difference by a factor that depends on how badly the model’s current preferences disagree with the Bradley-Terry model. If the model currently gives higher “implicit reward” or log-probability to the loser than to the winner, the mismatch is large, and the logistic factor is large in magnitude. That produces a big gradient step that strongly corrects the ranking. When the model already prefers the winner, the factor is small, so updates are smaller.

In the toy Gaussian diffusion example, the forward process has X_t distributed as $N(0, \sigma^2 + t)$ because the variance does not change over time; the process rescales the noise at each step so that all added noise cancels out, leaving the original variance plus a constant offset t regardless of how many steps have passed.

If you do reverse diffusion using only the conditional mean and no randomness, each noisy point deterministically moves along a single most likely trajectory back toward the data manifold. Different noise realizations that should end up spread out instead get collapsed onto similar paths. Over many steps this squeezes the distribution, so the final samples are too concentrated: the variance of the generated distribution ends up smaller than the true data variance.

To create training data for the denoiser $f(x_t, t)$:

- Sample a clean data point x_0 from the dataset.
- Sample a timestep t from some schedule (often uniformly from 1 to T).
- Run the forward noising process to produce a noisy version x_t of x_0 at time t .

Then you train the network with input (x_t, t) and a target such as the actual noise that was added (noise-prediction training) or the original clean x_0 (clean-prediction training). In both cases the model learns to map a noisy sample and its time step to something that lets you step backward in the diffusion process.

In DDPM, the reverse process is stochastic: at each time step you sample from a Gaussian whose mean is given by the denoiser and whose variance is set by the diffusion schedule, so you follow a noisy reverse Markov chain. In DDIM, the reverse process is designed so that you can make it deterministic (or use correlated noise): instead of sampling new noise at each step, you follow a deterministic trajectory that still matches the same forward marginals, allowing fewer, larger steps and a one-to-one mapping from initial noise to final sample.

These are my (enter-separated) answers. Review them - help with any concepts i'm wrong

You're in really good shape on this stuff. Almost everything you wrote is *spot on*. I'll just go through and flag the few spots that are imprecise or slightly off, especially for diffusion.

Transformers / Attention

1. Softmax scaling

We divide the dot product QK^T by the square root of d_k ...

✓ Totally correct. You mentioned:

- dot products grow with d_k
- softmax saturates
- gradients get tiny/unstable
- scaling keeps logits $O(1)$

That's exactly the story.

2. Argmax attention

- a) ... single corresponding value vector
- b) ... argmax non-differentiable, gradients ~ 0

✓ Perfect. The key points:

- Output is a **hard lookup** of one value.
- Gradient is basically zero a.e. w.r.t. scores/weights, so attention can't be trained end-to-end in the usual way.

You nailed both.

3. Incremental decoding and caching

prev_K and prev_V store keys/values for previous tokens ... avoid recomputing ...

✓ Exactly right. The important bits:

- They store the whole prefix.
 - You only compute K/V for the **new** token.
 - This turns “recompute $O(n)$ ” into “update $O(1)$ ” per step (ignoring attention itself).
-

4. Multi-Query Attention (MQA)

... shared keys/values as common memory, diversity from queries ...

✓ Perfect intuition.

- Different heads = different “questions” about the **same** memory.
- Memory savings from not storing K/V per head.

Nothing to fix here.

RLHF / DPO

5. RLHF pipeline

SFT → Reward model → RL fine-tuning w/ KL penalty

✓ Very clear and correct. You captured:

- what each phase *does*
 - why: alignment with human preferences, preserve language quality via KL.
-

6. Bradley-Terry model motivation

latent score $r(x, y)$; probability depends on score difference; logistic link

✓ Good. You hit:

- dependence on difference
- symmetry
- logistic as a nice smooth probability mapping.

That’s exactly the BT story.

7. Partition function cancellation / DPO motivation

ratios of probabilities; $Z(x)$ cancels when comparing pair

✓ Correct. That's the technical heart of DPO: preferences can be expressed purely in terms of $\log(\pi/\pi_{\text{ref}})$, no partition function.

8. Gradient term in DPO

gradient of $\log \pi(\text{winner})$ minus gradient of $\log \pi(\text{loser}) \rightarrow$ increase winner prob, decrease loser prob

✓ Yes. That's literally "push probability mass from loser to winner."

9. Why DPO gives bigger updates when the model prefers the loser

logistic factor larger when model ranks loser above winner $\rightarrow$ big gradient

✓ Conceptually right. Tiny nuance for your mental model:

- The scalar factor is something like $\sigma(\hat{r}_\theta(y_l) - \hat{r}_\theta(y_w))$.
- If model is **very wrong** (loser » winner), this difference is large positive $\rightarrow \sigma \approx 1 \rightarrow$ **maximum update**.
- If model is already correct (winner higher), difference negative $\rightarrow \sigma$ small $\rightarrow$ tiny update.

That matches what you said.

Diffusion

Here's where there's one real mistake and everything else is basically fine.

10. Forward process distribution

You wrote:

In the toy Gaussian diffusion example, the forward process has X_t distributed as $N(0, \sigma^2 + t)$ because the variance does not change over time; the process rescales the noise at each step so that all added noise cancels out, leaving the original variance plus a constant offset t regardless of how many steps have passed.

There are two issues here:

1. **Contradiction:** You said "variance does not change over time" but also " $N(0, \sigma^2 + t)$ " (which *does* change with t).
2. **"Rescales noise so it cancels out"** – that's not what's going on in this toy diffusion.

Correct picture:

- Start with $X_0 \sim \mathcal{N}(0, \sigma^2)$.

- The forward process adds **independent Gaussian noise increments**. In continuous time, this is like:

$$X_t = X_0 + W_t,$$

where $W_t \sim \mathcal{N}(0, t)$ is a standard Brownian motion increment.

- Because X_0 and W_t are independent Gaussians:

$$\text{Var}(X_t) = \text{Var}(X_0) + \text{Var}(W_t) = \sigma^2 + t.$$

So:

- The variance **does** change over time — it **increases linearly in t**.
- Nothing is “cancelled”; the variances of the noise increments just **add**.

The part “ $\mathcal{N}(0, \sigma^2 + t)$ ” was right; just update the explanation in your head to “sum of independent Gaussians → variances add.”

11. Reverse diffusion with only conditional mean

deterministic path → different noise realizations collapse → smaller variance than true data

✓ This is right conceptually.

If you want the precise toy math:

- Each reverse step is: $X_{t-\Delta t} = a_t X_t$ (no noise).
- After many steps: $X_0 = (\prod_t a_t) X_1$.
- So
 $\text{Var}(X_0) = (\prod_t a_t)^2 \text{Var}(X_1)$,
 and that product is <1 , often **much** <1 → variance shrinks too much.

That’s what they show with the σ^4 vs σ^2 .

12. Training data for the denoiser

sample x_0 , sample t , run forward process to get x_t , input (x_t, t) , target noise or x_0

✓ Exactly the standard diffusion training loop. You clearly know both noise-prediction and x_0 -prediction views.

13. DDPM vs DDIM

DDPM: noisy reverse chain

DDIM: deterministic trajectory (or correlated noise), same marginals, fewer larger steps

✓ This is a nice, clean conceptual summary.

TL;DR: What to fix in your head

Only **one real conceptual correction** you should internalize:

- **Forward diffusion variance:**
 - It does **not** “stay constant” or “cancel out.”
 - It **grows** as $\sigma^2 + t$ because you add independent Gaussian noise over time, and **variances add**.

Everything else you wrote is accurate and at “review-session ready” level.

If you want, I can now give you:

- a 5–10 line *super* compressed crib sheet for each of: **MQA vs MHA**, **DPO derivation**, and **DDPM vs DDIM** that you can glance at in the hallway between review sections.

This tool was fairly accurate in its descriptions. Going through Sultan's discussion right now, the topics were explained clearly enough for me to understand. I also intentionally gave it a wrong answer in my answers to its' generated questions, and it pointed it out to me, which was nice.